Intent-Aware Hybrid Retrieval System

30-Day Project Plan • RAG + LangChain • HCL Hack60

Project	Intent-Aware & Explainable Hybrid Retrieval System
Duration	30 Days (4 Weeks)
Core Stack	Python 3.10, LangChain, FAISS, OpenSearch, Neo4j, FastAPI
LLM Layer	Sentence-BERT, Cross-Encoder Re-ranker, Ollama/OpenAI
Evaluation	RAGAS (Context Precision, Recall, Faithfulness, Answer Relevancy)
Final Output	Web App + REST API + Knowledge Graph + Explainability Layer

This document is a step-by-step implementation guide with inline testing checkpoints after every major task so you can verify progress continuously.

WEEK 1 — Foundation & Data Pipeline (Days 1–7)

■ Day 1–2 : Environment Setup

- Create virtual environment: python -m venv venv && source venv/bin/activate
- Install core packages: pip install langchain langchain-community sentence-transformers
- Install retrieval stack: pip install faiss-cpu opensearch-py neo4j
- Install extras: pip install fastapi uvicorn ragas spacy torch transformers
- Start Docker services: docker run -p 9200:9200 opensearchproject/opensearch:2.11.0
- Start Neo4j: docker run -p 7474:7474 -p 7687:7687 neo4j:5
- Create project structure: /data /embeddings /retrieval /reranker /graph /api /eval
- Set up Git repository with main and dev branches

```
# Quick environment smoke test
import langchain, faiss, sentence_transformers
print('All imports OK')
```

■ TESTING CHECKPOINTS:

✔ Run: python -c "import langchain; print(langchain.__version__)" — should print version

✔ Run: curl http://localhost:9200 — OpenSearch should return JSON cluster info

✔ Open: http://localhost:7474 — Neo4j Browser should load

✔ Run: python -c "from sentence_transformers import SentenceTransformer; print('OK')"

✔ Confirm folder structure exists with: ls -la project/

■ Day 3–4 : Dataset Selection & Preprocessing

- Download dataset: Indian Legal docs / Supreme Court judgments (or any domain corpus)
- Load raw text files and inspect structure (encoding, length, format)
- Clean text: remove HTML tags, fix UTF-8 encoding, strip boilerplate headers/footers
- Implement RecursiveCharacterTextSplitter (chunk_size=512, chunk_overlap=50)
- Store chunks in SQLite with columns: id, source, page, chunk_text, metadata_json
- Log chunk count, average chunk length, and outlier chunks (>800 or <50 tokens)

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(chunk_size=512, chunk_overlap=50)
chunks = splitter.split_text(raw_text)
print(f'Total chunks: {len(chunks)}')
```

■ TESTING CHECKPOINTS:

✔ Assert total chunks > 1000 for meaningful retrieval testing

✔ Print: avg_chars, min_chars, max_chars per chunk — no chunk should be empty

✔ Spot-check 5 random chunks manually for quality

✔ Verify SQLite DB: SELECT COUNT(*) FROM chunks — should match expected number

✔ Check no chunk contains raw HTML tags or encoding errors (\x escapes)

■ Day 5–6 : Embedding Pipeline

- Load Sentence-BERT model: all-MiniLM-L6-v2 (fast) or all-mpnet-base-v2 (accurate)
- Batch-encode all chunks (batch_size=64) and save embeddings as .npy file
- Build FAISS IndexFlatL2 index, add all vectors, save index to disk
- Set up OpenSearch index with BM25 analyzer and bulk-index all chunk texts
- Write helper functions: embed_query(), search_faiss(), search_opensearch()

```
import faiss, numpy as np
index = faiss.IndexFlatL2(384)
index.add(embeddings)
D, I = index.search(query_vec, k=5) # D=distances, I=indices
print('Top indices:', I[0])
```

■ TESTING CHECKPOINTS:

✔ Test FAISS: query 'legal judgment privacy' → top-5 results printed with scores

✔ Test OpenSearch BM25: same query → top-5 results should overlap partially with FAISS

✔ Verify embedding shape: assert embeddings.shape == (num_chunks, 384)

✔ Check FAISS index size: index.ntotal should equal number of chunks

✔ Time a single query: should complete in < 200ms for FAISS

■ Day 7 : Pipeline Review & Buffer

- Run full pipeline: raw text → chunks → embeddings → FAISS + OpenSearch query
- Fix any bugs found in Days 1–6
- Write README section: Setup Instructions
- Commit all working code to Git with tag: v0.1-data-pipeline

■ TESTING CHECKPOINTS:

✔ End-to-end test: input a raw document, get top-5 search results — full flow working

✔ Git log shows clean commits for every day

✔ README is readable and setup can be reproduced from scratch

---

WEEK 2 — Core Retrieval System (Days 8–14)

■ Day 8–9 : Hybrid Search with RRF Fusion

- Wrap FAISS in LangChain FAISS vectorstore with InMemoryDocstore
- Wrap OpenSearch as LangChain retriever using OpenSearchVectorSearch
- Implement Reciprocal Rank Fusion: score = Σ 1/(k + rank_i) where k=60
- Combine both retrievers using LangChain EnsembleRetriever(weights=[0.5, 0.5])
- Test with 20 sample queries and compare hybrid vs individual retriever results

```
from langchain.retrievers import EnsembleRetriever
retriever = EnsembleRetriever(
retrievers=[bm25_retriever, faiss_retriever],
weights=[0.5, 0.5])
results = retriever.get_relevant_documents('privacy judgment India')
```

■ TESTING CHECKPOINTS:

✔ Run same query on BM25-only, FAISS-only, and Hybrid — hybrid should have best top-3

✔ Check that results from both retrievers are merged and deduplicated

✔ Verify RRF scores are assigned correctly (higher-ranked items get higher scores)

✔ Assert: len(hybrid_results) >= max(len(bm25_results), len(faiss_results))

✔ Manual review: are top-3 results actually relevant to query?

■ Day 10–11 : Intent Detection & Query Decomposition

- Build LangChain PromptTemplate for intent classification (factual / exploratory / navigational)
- Build LangChain chain for entity/filter extraction: date, domain, entity type
- Build query decomposition chain: one complex query → 2-4 simpler sub-queries
- Chain together: raw query → intent + filters + sub-queries → structured dict output
- Use Ollama (local) or OpenAI as the LLM backbone

```
from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain
prompt = PromptTemplate(input_variables=['query'],
template='Classify intent of: {query}. Return: factual/exploratory/navigational')
chain = LLMChain(llm=llm, prompt=prompt)
result = chain.run('What is Article 21?')
```

■ TESTING CHECKPOINTS:

✔ Test intent: 'What is Section 377?' → should return 'factual'

✔ Test intent: 'Explore privacy laws in India' → should return 'exploratory'

✔ Test filter extraction on 'Recent Supreme Court cases 2023' → {year: 2023, domain: legal}

✔ Test decomposition: 'Latest privacy judgments after 2020' → ['privacy judgments', 'cases after 2020']

✔ Check structured output is valid Python dict (no JSON parse errors)

■ Day 12–13 : Multi-Query Retrieval

- Implement LangChain MultiQueryRetriever wrapping the hybrid EnsembleRetriever
- Configure LLM to generate 4 query variants for each original query
- Retrieve results for all variants and merge/deduplicate using unique doc IDs
- Implement a UniqueDocumentFilter to remove duplicates by content hash
- Compare recall@10 before and after multi-query (should improve by ~15-25%)

```
from langchain.retrievers.multi_query import MultiQueryRetriever
mq_retriever = MultiQueryRetriever.from_llm(
retriever=hybrid_retriever, llm=llm)
docs = mq_retriever.get_relevant_documents('privacy rights India')
print(f'Retrieved {len(docs)} unique docs')
```

■ TESTING CHECKPOINTS:

✔ For query 'privacy laws India': print all 4 generated variants — must be semantically different

✔ Assert: len(multi_query_results) > len(single_query_results) for most queries

✔ Check no duplicate documents in final result list (hash-based dedup test)

✔ Recall test: manually check if ground-truth doc appears in top-10 more often now

✔ Time test: full multi-query retrieval should complete in < 3 seconds

---

■ Day 14 : Benchmarking Round 1

- Create a test set of 50 queries with known relevant documents (ground truth)
- Compute Precision@5, Precision@10, Recall@10 for hybrid retriever
- Log results in a CSV: query, precision_5, recall_10, top_result_relevant (Y/N)
- Identify bottom-10 worst performing queries for targeted improvement
- Commit benchmark baseline to Git with tag: v0.2-hybrid-retrieval

■ TESTING CHECKPOINTS:

✔ Precision@5 should be > 0.60 (at least 3 of top 5 results are relevant)

✔ Recall@10 should be > 0.70 (at least 70% of relevant docs appear in top 10)

✔ CSV file is correctly written and can be loaded with pandas

✔ Worst-10 queries list is reviewed and root causes noted (bad chunking? wrong intent?)

WEEK 3 — Knowledge Graph + Re-Ranking + Full RAG (Days 15–21)

■ Day 15–16 : Knowledge Graph with Neo4j

- Design graph schema: Nodes (Document, Person, Case, Law, Date), Edges (MENTIONS, RELATED_TO, CITES)
- Extract named entities using spaCy (en_core_web_sm) or an LLM NER chain
- Write Neo4j ingestion script using official neo4j Python driver
- Create MERGE queries to avoid duplicate nodes
- Build LangChain GraphCypherQAChain with natural-language-to-Cypher translation

```
from neo4j import GraphDatabase
driver = GraphDatabase.driver('bolt://localhost:7687', auth=('neo4j','password'))
with driver.session() as s:
s.run('MERGE (d:Document {id: $id})', id='doc_001')
print('Graph node created')
```

■ TESTING CHECKPOINTS:

✔ Run: MATCH (n) RETURN count(n) in Neo4j Browser — should show > 500 nodes
✔ Run: MATCH ()-[r]->() RETURN count(r) — should show > 1000 relationships
✔ Test Cypher query: 'Find all cases related to Article 21' via GraphCypherQAChain
✔ Verify entity extraction: 5 sample chunks should yield correct Person/Law entities
✔ Check MERGE logic: re-running ingestion should not create duplicate nodes

---

■ Day 17 : Graph-Augmented Retrieval

- After initial vector search, extract entities from top-5 results
- Query Neo4j: find documents RELATED_TO or CITES the retrieved documents
- Merge graph-found docs with vector results using RRF (give graph results weight 0.3)
- Build a unified retrieval pipeline: Vector → Graph Expansion → RRF Merge
- Test on queries that require relationship traversal (e.g. cases citing a specific law)

■ TESTING CHECKPOINTS:

✔ Test: query 'cases citing Article 21' → graph should surface more related docs than vector alone
✔ Verify: graph-expanded results include at least 1 document not in pure vector results
✔ Check merge pipeline doesn't crash when Neo4j returns 0 related nodes
✔ Performance test: augmented retrieval should complete in < 5 seconds end-to-end
✔ Manual review: are the graph-added documents actually relevant?

■ Day 18–19 : Neural Re-Ranking with Cross-Encoder

- Load cross-encoder: cross-encoder/ms-marco-MiniLM-L-6-v2 from HuggingFace
- Implement rerank(query, docs, top_k=10) function using cross-encoder scores
- Pipeline: Hybrid Retrieval (top-50) → Cross-Encoder Re-rank → top-10 final results
- Wrap as a LangChain custom BaseRetriever subclass
- Test re-ranking on 20 queries and compare order before/after

```
from sentence_transformers import CrossEncoder
ce = CrossEncoder('cross-encoder/ms-marco-MiniLM-L-6-v2')
pairs = [(query, doc.page_content) for doc in candidate_docs]
scores = ce.predict(pairs)
ranked = sorted(zip(scores, candidate_docs), reverse=True)[:10]
```

■ TESTING CHECKPOINTS:

✔ Test: top result after re-ranking should be more relevant than top result before
✔ Assert: re-ranked results have higher avg cross-encoder score than original top-10
✔ Verify: top-10 from re-ranking is a subset of top-50 from retrieval (no new docs added)
✔ Speed test: cross-encoder re-ranking of 50 docs should complete in < 2 seconds on CPU
✔ Check for model loading errors and OOM issues on your machine

---

■ Day 20–21 : Full RAG Pipeline with LangChain

- Assemble complete RAG chain: Intent → Multi-Query → Hybrid → Graph → Re-rank → LLM
- Use LangChain RetrievalQA or create a custom LCEL (LangChain Expression Language) chain
- Add source citations to every answer: each claim linked to a source document
- Implement conversation memory using ConversationBufferWindowMemory (last 5 turns)
- Add answer post-processing: format, clean, add confidence indicator

```
from langchain.chains import RetrievalQA
qa_chain = RetrievalQA.from_chain_type(
llm=llm,
chain_type='stuff',
retriever=full_retriever,
return_source_documents=True)
result = qa_chain({'query': 'What is Article 21?'})
print(result['result']) # answer
print(result['source_documents']) # citations
```

■ TESTING CHECKPOINTS:

✔ Ask 10 test questions — all must return a coherent answer with source citations

✔ Multi-turn test: ask a follow-up question referencing the previous answer — context maintained

✔ Check that every answer includes at least 1 source document reference

✔ Test with an out-of-scope question — system should say 'Not found' gracefully

✔ Assert: answer generation time < 15 seconds including all retrieval steps

WEEK 4 — Explainability + Evaluation + API/UI (Days 22–30)

■ Day 22–23 : Explainability Layer

- For each result, generate: 'Why retrieved?' using LLM chain with retrieval metadata
- Show which retrieval method contributed: BM25 / Semantic / Graph (with % weight)
- Display confidence breakdown: BM25 score, cosine similarity, cross-encoder score
- Highlight matching terms between query and result using simple token overlap
- Generate 1-sentence human-readable explanation per result using LLM

■ TESTING CHECKPOINTS:

✔ Test: each result must have a non-empty 'explanation' field
✔ Check retrieval source tags are correct (BM25-found doc should NOT say 'Semantic')
✔ Verify confidence scores are in [0,1] range and sorted correctly (higher = more relevant)
✔ Human review: does the 1-sentence explanation accurately describe why the doc is relevant?
✔ Test with 10 queries — all explanations must be grammatically correct

---

■ Day 24–25 : RAGAS Evaluation Suite

- Create evaluation dataset: 100 (question, ground_truth_answer, relevant_contexts) triplets
- Run RAGAS metrics: context_precision, context_recall, answer_relevancy, faithfulness
- Log all scores to a DataFrame and save as eval_results.csv
- Iterate: if context_recall < 0.80, tune chunk size or retrieval top-k
- If faithfulness < 0.75, update prompt to be more grounded in retrieved context

```
from ragas import evaluate
from ragas.metrics import context_precision, context_recall,
answer_relevancy, faithfulness
from datasets import Dataset
results = evaluate(dataset, metrics=[context_precision,
context_recall, answer_relevancy, faithfulness])
print(results) # prints score table
```

■ TESTING CHECKPOINTS:

✔ All 4 RAGAS metrics must run without errors on the full 100-query eval set
✔ Target thresholds: Context Recall > 0.80, Faithfulness > 0.75, Answer Relevancy > 0.75
✔ eval_results.csv is correctly saved with all scores
✔ Identify bottom-10 worst queries and fix at least 5 of them through tuning
✔ Final RAGAS scores after tuning should be >= baseline from before tuning

---

■ Day 26–27 : FastAPI Backend

- Build POST /search endpoint: accepts {query, filters} → returns {results, explanations}
- Build GET /explain/{doc_id}: returns full explanation for a specific result
- Build GET /graph/{entity}: returns Neo4j neighborhood of an entity
- Build POST /feedback: stores user thumbs-up/down for future fine-tuning
- Add async support with async def + await for all I/O-bound operations
- Add input validation with Pydantic models and error handling with HTTPException

```
from fastapi import FastAPI
app = FastAPI()
@app.post('/search')
async def search(query: str, filters: dict = {}):
results = await run_rag_pipeline(query, filters)
return {'results': results}
# Run: uvicorn main:app --reload
```

■ TESTING CHECKPOINTS:

✔ Test POST /search with curl or Postman: must return JSON in < 10 seconds
✔ Test GET /explain/{id}: valid ID returns explanation; invalid ID returns 404
✔ Test POST /feedback: verify entry is stored in DB
✔ Load test: 5 concurrent requests to /search should all complete successfully
✔ Open http://localhost:8000/docs — FastAPI Swagger UI must show all endpoints

---

■ Day 28–29 : Web App Frontend

- Build Streamlit frontend (faster) OR React app (preferred per problem statement)
- Search bar with real-time suggestions as user types
- Results panel: each result shows title, snippet, explanation badge, source tag, confidence score
- Filter sidebar: date range, domain, document type
- Knowledge graph visualization using vis.js (HTML component in Streamlit) or react-force-graph
- Feedback buttons (thumbs up/down) on each result

■ TESTING CHECKPOINTS:

✔ Search from UI returns results in < 10 seconds and displays them correctly
✔ Filter by year: only docs from that year should appear in results
✔ Knowledge graph renders without JS errors in browser console
✔ Feedback button click stores feedback (verify in DB)
✔ Test on mobile screen size (responsive layout check)

■ Day 30 : Final Testing, Polish & Submission

- Run full end-to-end test on 20 fresh queries not seen during development
- Verify 95%+ retrieval rate: at least 19/20 queries return a relevant result
- Write full README: project overview, setup steps, architecture diagram, how to run
- Create architecture diagram showing all 5 pipeline stages
- Record 3-5 minute demo video showing search, explanation, graph visualization
- Package as GitHub repo with requirements.txt, docker-compose.yml, and sample data
- Final Git tag: v1.0-submission

■ TESTING CHECKPOINTS:

✔ 20/20 queries return results (no crashes or empty responses)
✔ At least 19/20 top results are relevant (95% target met)
✔ docker-compose up brings the entire system up from scratch
✔ All RAGAS metrics meet minimum thresholds on final evaluation run
✔ Demo video plays correctly and shows all key features
✔ GitHub repo is public and README is clear enough for a stranger to run it

---

Technology Stack Summary

Layer	Technology / Library
Embeddings	sentence-transformers (all-MiniLM-L6-v2 / all-mpnet-base-v2)
Vector Store	FAISS (dense), OpenSearch (BM25 sparse)
LLM Orchestration	LangChain (chains, retrievers, memory, prompts)
LLM Backend	Ollama + Mistral/LLaMA3 (local) or OpenAI GPT-4o
Knowledge Graph	Neo4j 5 + neo4j Python driver + LangChain GraphCypherQAChain
Re-Ranking	HuggingFace CrossEncoder (ms-marco-MiniLM-L-6-v2)
Evaluation	RAGAS (context_precision, context_recall, faithfulness, relevancy)
API Layer	FastAPI + Pydantic + Uvicorn (async)
Frontend	Streamlit (fast) or React + Tailwind (preferred)
Graph Visualization	vis.js / react-force-graph / Cytoscape.js
Database	SQLite (chunks) / PostgreSQL (production)
Containerization	Docker + docker-compose for OpenSearch & Neo4j

Pro Tips for Success

★ Use Ollama locally (ollama pull mistral) to avoid OpenAI API costs during development.

★ Cache all embeddings to .npy files — recomputing wastes hours every restart.

★ Start RAGAS evaluation from Week 2 on small batches, not just at the end.

★ Keep a daily_log.md — note what worked, what failed, and your benchmark scores each day.

★ For Neo4j ingestion, always use MERGE not CREATE to prevent duplicate node explosions.

★ Set retrieval top-k=50 for cross-encoder input but return only top-10 to the user.

★ Test edge cases: empty query, very long query (>200 words), queries in Hindi/mixed language.

★ Use LangChain's LCEL (pipe operator |) for cleaner chain composition in Week 3+.

Final Success Metrics (Submission Targets)

Metric	Minimum Target	Stretch Goal
Retrieval Rate	95% queries	98% queries
Context Recall (RAGAS)	> 0.80	> 0.88
Context Precision	> 0.75	> 0.85
Answer Faithfulness	> 0.75	> 0.85
Answer Relevancy	> 0.75	> 0.85
API Response Time	< 10 seconds	< 5 seconds
Re-ranking Improvement	+10% Precision@5	+20% Precision@5

Good luck with your Hack60 submission! Follow the plan day by day, run every test checkpoint, and you will have a working system well before Day 30.